BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION Sisteme informatice avansate

DISSERTATION THESIS

A Cloud-Based Closed-Loop Payment System with Hybrid On-Chip Reconciliation for Festival Environments

Supervisor
Assoc. Prof. Christian Sacarea, PhD

Author
Tudor Esan

2026

ABSTRACT

This thesis describes a cloud-based closed-loop payment system for music festivals and large outdoor events. The system is online by default and uses the bracelet balance as a fallback when the network goes down. Festivals are known for bad connectivity when a lot of people gather in the same place, but in our view, building the whole system around the assumption that the network will fail brings more complexity than it is worth. We keep things online most of the time using a Starlink uplink and a redundant network topology, and we let the NFC wristband hold the balance as a cache for the rest. When the network is up, transactions go through the server in real time. When it is not, the chip itself stores the balance and a monotonic debit counter, so vendor terminals can approve a payment locally and not let anyone spend money twice. As soon as connectivity comes back, the terminals push their queued transactions to the server in batches, and the server applies them against its own credit counter. Credits are written only by the server and debits are written only by the chip, so the two flows meet again on their own when the network is back. We assess the architecture against what a real festival actually needs and we argue that treating offline as a bounded fallback, and not as the foundation of the system, gives a simpler and more maintainable result.

Contents

1 Introduction **1**

1.1 Problem Statement 2

1.2 Objectives 4

1.3 Related Work 5

2 Background and Related Work **7**

2.1 Closed-Loop Payment Systems 7

2.1.1 Transit Card Systems 9

2.1.2 Festival Payment Systems 12

2.1.3 EMV and Offline Authorization 14

2.2 Offline-First Architecture Patterns 15

2.2.1 Local-First Software 16

2.2.2 Conflict-Free Replicated Data Types (CRDTs) 17

2.2.3 Event Sourcing and CQRS 18

2.2.4 Edge Computing 18

2.3 Conflict Resolution in Distributed Systems 19

2.3.1 The CAP Theorem 19

2.3.2 Eventual Consistency 20

2.3.3 Transaction Ordering and Why Timestamps Are Enough . . . 21

3 System Architecture **23**

3.1 High-level overview 23

3.1.1 The hybrid reconciliation model 24

3.2 Balance reconciliation model 25

3.2.1 State on each side 25

3.2.2 Invariants 25

3.2.3 The four operations 26

3.2.4 Worked example 26

3.2.5 Lost transactions 27

3.3 Lessons from related systems and patterns 27

3.3.1 Offline operation 28

3.3.2 Top-ups: the one thing that needs the network 28
3.4 The mobile application 28
3.4.1 Attendee view 28
3.4.2 Vendor POS mode 29
3.5 Admin dashboard 29
3.6 Backend server 29
3.6.1 Multi-tenancy 29
3.6.2 Horizontal scalability 30
3.7 Database: PostgreSQL 30

4 Implementation **32**
4.1 Implementation overview 32
4.2 Database schema for the hybrid model 33
4.3 The reconciliation service and sync endpoint 34
4.4 Chip data layout and HMAC 36
4.5 Online debit path 37
4.6 Offline debit path and the queue 38
4.7 Topup path 39

5 Security Considerations **40**
5.1 Authentication and Authorization 40
5.2 Wallet Integrity and Double-Spend Prevention 40
5.2.1 Online Mode 40
5.2.2 Offline Mode 40
5.3 Offline Trust Model 40
5.4 Stripe and Payment Security 40
5.5 Data Protection 40

6 Discussion **41**
6.1 Design Trade-offs 41
6.1.1 Online by default, offline as a fallback 41
6.1.2 Server balance versus chip balance 42
6.1.3 Consistency versus availability during outages 42
6.1.4 Lost transactions and operator risk 43
6.1.5 Starlink and a redundant topology versus other connectivity options 43
6.2 Comparison with Existing Solutions 43
6.2.1 Commercial festival platforms 43
6.2.2 Transit cards 44
6.2.3 Open versus proprietary 45

6.3 Limitations 45
6.3.1 Hardware: a plain NFC chip is not enough for production . . 45
6.3.2 Offline duration is assumed to be short 46
6.3.3 Single-event scope in testing 46
6.3.4 No peer-to-peer transfers between attendees 46
6.3.5 Refund flow is partial 46
6.3.6 No real load testing 46
6.3.7 Vendor onboarding is a manual review 47
6.4 Further Improvements 47
6.4.1 Move to authenticated NFC chips 47
6.4.2 Fraud detection on the server 47
6.4.3 Better offline strategies 47
6.4.4 Real-time analytics for the operator 48
6.4.5 Multi-event support and tenant isolation 48
6.4.6 Vendor settlement and payout automation 48
6.4.7 Integration with other festival systems 48

7 Conclusions **49**

Bibliography **50**

Chapter 1

Introduction

Music festivals and large outdoor events have become a massive part of modern culture. Every summer, millions of people gather in open fields, parks, and temporary venues to enjoy live performances and food. From Untold in Romania to Tomorrowland in Belgium, these events keep growing in size and complexity. But behind all the excitement, there is a very practical problem that organizers have to deal with: how do you let tens of thousands of people buy food, drinks, and merchandise in a place where the internet barely works?

The traditional approach was straightforward: people brought cash, stood in line, and paid at the counter. It was slow, it was messy with always having change and there was always the risk of theft or losing your wallet in the crowd. Over the past decade, many festivals have moved towards cashless payment systems, typically based on RFID wristbands or prepaid cards. The idea is simple: you load money onto your wristband before or during the event, and then you just tap it at a vendor terminal to pay. Industry data from cashless technology providers suggests that RFID systems reduce transaction times by up to 80 percent compared to cash and that festival organizers see average revenue increases of around 22 percent after going cashless [Tap24]. These figures are consistent with the academic evidence: a recent meta-analysis across 71 studies found a statistically significant, small positive effect of cashless payments on consumer spending ($g = 0.135$, 95% CI [0.068; 0.203]), driven primarily by the reduced psychological friction of not physically handing over money [SBH24].

However, these systems come with a critical weakness: most of them depend on a reliable internet connection. And if there is one thing that festivals are known for, it is terrible connectivity. When tens of thousands of people gather in the same area, all using their phones simultaneously, the cellular network simply cannot handle the load. Shafiq et al. [SJL+13] conducted the first large-scale characterization of cellular network performance during crowded events using real-world traces from a

tier-1 US network. Their findings are impressive: pre-connection failures increased by 100 to 5,000 times compared to routine days, driven by users exhausting the limited bandwidth of the signaling channel when too many devices attempt to acquire radio resources simultaneously. This degradation extended as far as 10 miles around the event venue as attendees arrived and departed. Even for users who managed to connect, voice call failures increased by 7 to 30 times, packet, and round-trip times increased by up to 7 times. The performance spikes were particularly severe during intermissions, when large numbers of attendees simultaneously attempted to use their devices. Notably, the study found that even with portable base stations and free Wi-Fi offloading already deployed, these degradation levels persisted, highlighting a fundamental capacity limitation rather than a simple configuration problem.

This is where things get problematic for payment systems. If the payment terminal at a food stall needs to contact a remote server to authorize a transaction, and the network is down or painfully slow, the transaction fails. The attendee cannot buy food. The vendor loses a sale. Multiply this by tens of thousands of transactions per day, and you can see why this is a serious issue. The most famous example of this going wrong happened at the Download Festival in the UK in 2015, the first major UK festival to go completely cashless. The RFID system experienced technical failures that left attendees unable to purchase food or drinks, with many taking to social media to describe the system as useless. Although organisers claimed only around one percent of attendees were affected, the backlash was severe enough that the festival abandoned the cashless-only approach the following year [eFe16].

The core question this thesis tries to answer is: can we build a payment system that works reliably at festivals and that does not fall apart when the network has a bad moment? Our approach is to design a system that operates online by default, with connectivity coming from a Starlink uplink and a redundant network topology, so that vendor terminals can talk to the cloud backend in real time most of the time. But no network is perfectly reliable, especially one assembled in a field for a few days, so we also need a fallback for the moments when the network is down. Instead of building a separate offline server or a heavy synchronization layer, we let the NFC wristband itself act as the cache

1.1 Problem Statement

Festivals put together a few problems that, on their own, payment systems already know how to solve, but putting them all together its when it becomes tricky. The first one is connectivity. Cellular networks fall over when too many people gather in the same place, and many festivals happen in fields and forests where there is no proper

wired infrastructure to begin with. Our answer is to bring our own connectivity to the venue: a Starlink uplink with a redundant network topology, so that the vendor terminals can talk to the cloud backend most of the time. When the network is up, transactions are authorized in real time, wallet balances are fresh, and operations that need an external provider like Stripe work as you would expect.

A network we set up ourselves is still a network, and it can still go down. Equipment fails, the density of wireless devices in a small area creates dead zones, and multi-day events keep stressing infrastructure that was put together in a field a few days before. When the network does drop for a few minutes during a peak hour, the system still has to take payments, otherwise people cannot buy a sandwich and the whole festival economy stops. So we need a fallback that takes over when the network is gone and gets out of the way when it comes back. We do not want to design the whole system around the assumption that the network will fail, because that would push complexity everywhere for a problem that is online most of the time.

The second piece is that this is a closed economy. Unlike a normal shop where you tap a bank card and the bank takes care of authorization, a festival has its own internal currency: attendees load tokens, spend them at approved vendors, and get refunded for what they did not use. The money never leaves the perimeter of the event. This is actually an advantage for us, because we control the whole flow and we are not blocked by an external authorization network that we could not reach during an outage anyway.

The third piece is scale and speed. A medium festival can have hundreds of vendors processing thousands of transactions per hour, with very sharp peaks around meals and between concerts. Nobody is going to wait two minutes for a payment to go through. Online, this is just a matter of provisioning the backend to handle the spike. Offline, the question becomes how to keep the same speed at each terminal without giving up correctness, which is the next problem.

The fourth piece is consistency during offline windows. If two terminals lose the network at the same time and both approve a payment from the same wallet, they could together let through more money than is actually there. This is the well-known double-spend problem, and it is pretty hard in distributed systems. Refusing the payment is not an option for us, because at a festival "we cannot pay because the Wi-Fi is bad" is not an acceptable answer. So we choose availability, and we make sure the architecture limits how badly things can go wrong: each wristband carries its own balance and a debit counter on the chip itself, so the chip is the one that decides locally whether a payment fits, and double-spend across terminals is not possible because the chip is a single physical object that the customer carries between vendors. If minor discrepancies arrise the festival will cover the loss.

The fifth piece is reconciliation. When terminals come back online, the transactions they processed offline have to land in the server in a way that does not lose anything and does not double-apply anything. Because we are online-first, these windows are expected to be short and not that frequent, but even a few minutes of outage during a peak can produce hundreds of queued transactions per terminal, and the sync has to be idempotent and order-tolerant.

One last thing worth saying out loud: not every operation can work offline. Wallet top-ups in particular need real-time communication with Stripe (or any other on ramp provider) and with our backend, because real money is moving from the attendee's bank account into the system. You cannot process that on the chip. This is on purpose: the chip handles debits when the network is gone, but credits, refunds, and anything that touches external money stay strictly online and stay strictly on the server.

1.2 Objectives

The main objective of this thesis is to design and implement a minimum viable product (MVP) for a cloud-based, closed-loop payment system tailored to festival and event environments. The system is online by default, with connectivity coming from a Starlink uplink and a redundant network topology, and uses the NFC wristband itself as a local cache so that vendor terminals can keep approving payments during short network outages. The concept of a minimum viable product, as defined by Ries [Rie11], refers to a version of a product that allows the team to collect the maximum amount of validated learning with the least effort. In our case, this means building enough of the system to demonstrate that the core architectural ideas work, without necessarily implementing every feature a production deployment would need. Throughout this work, we explore different design architectures and patterns, discussing their benefits and trade-offs in the context of festival environments, before settling on the approach that best fits the constraints of the problem.

Specifically, the objectives of this work are the following:

O1: Design a single, unified system architecture in which the cloud backend is the source of truth and the NFC wristband holds the balance and a debit counter on the chip itself. Payments are authorized in real time against the server when the network is up, and against the chip directly when the network is temporarily down. There is no separate offline server, no on-site infrastructure, and no mode switch in the application: the same code path reads the chip, decides whether the payment fits, and either talks to the server or queues the transaction for later sync.

O2: Implement a wallet-based payment service capable of processing debit and

credit operations on a per-bracelet, per-event balance. Wallet top-ups go through Stripe and stay online by design, since they involve real money entering the system. Debit transactions (payments at vendor terminals) are authorized against the server when connectivity is available, and against the chip's cached balance and debit counter when it is not, with the terminal queueing the transaction for batch sync once the network is back.

O3: Define a reconciliation model that keeps the chip-side debits and the server-side credits in two non-overlapping lanes, each with a single writer (the chip for debits, the server for credits), so that the two streams converge on the same balance once connectivity is restored, without any conflict resolution beyond per-transaction idempotency.

O4: Implement a core service that manages vendors, products, stock, users, and events, providing the administrative backbone of the system.

O5: Develop client applications for the three main user roles: a Consumer App for attendees to top up their wristband and make payments via NFC, a Vendor App for merchants to manage products and accept payments, and an Admin Dashboard for event organizers to oversee the entire operation.

O6: Evaluate the proposed architecture against the specific requirements of festival environments. The detailed discussion of trade-offs between the online path and the chip-based fallback, the limitations of the current implementation, and the reasoning behind treating the on-chip cache as a bounded fallback rather than the foundation of the system, is deferred to Chapter 6.

1.3 Related Work

The idea of building payment systems that work without constant connectivity is not new, but most of the existing work comes from domains that are quite different from festivals. Transit card systems like Suica, Oyster, and Octopus have been doing offline transactions for decades, storing balances directly on the card chip so that gates can open without waiting for a server response. These systems are well studied and provide useful lessons about transaction speed and security pitfalls, but they operate in a controlled infrastructure with fixed terminals and predictable usage patterns, which is not exactly what a festival looks like.

On the software architecture side, the local-first movement, particularly the work by Kleppmann et al. [KWvHM19], lays out principles for applications that treat the local device as the primary copy and synchronize with the cloud when they can. Concepts like Conflict-Free Replicated Data Types offer formal guarantees about convergence in distributed systems, and event sourcing provides a natural model for logging transactions that happened offline and replaying them later. These ideas

inform our design, though they do not map one-to-one onto payment systems, since money has the annoying property of being a conserved quantity.

On the commercial side, providers like Tappit, Intellitix, Weezevent, and Glownet have deployed cashless systems at hundreds of festivals, but their architectures are proprietary and essentially undocumented in academic literature. The failures we do know about, such as the Download Festival 2015 crash or the Danish Nets outage in 2024, highlight the consequences of getting this wrong, but they do not tell us much about how to get it right.

In short, the building blocks exist across transit systems, distributed systems theory, and commercial festival deployments, but nobody has put them together into a documented, open architecture designed for the specific constraints of festival environments. That is the gap this thesis tries to fill. A more detailed treatment of each of these areas follows in Chapter 2.

Chapter 2

Background and Related Work

This chapter provides the theoretical and practical background needed to understand the design decisions we make later in the thesis. We start by looking at closed-loop payment systems, which is the category our system falls into, and then move on to the architectural patterns that deal with offline operation and distributed data. We also review the existing commercial solutions for festival payments and discuss the academic literature on conflict resolution in distributed systems. The goal is to give the reader enough context to understand why we made the choices we did, and what alternatives we considered along the way.

2.1 Closed-Loop Payment Systems

Before going into the technical details, it is worth clarifying what we mean by a closed-loop payment system, since this concept is at the core of our architecture. In the payment industry, there are basically two types of systems: open-loop and closed-loop. An open-loop system is what most people use every day without thinking about it. When you pay with a Visa or Mastercard, the transaction goes through a network of banks, card schemes, and processors that connect merchants to customers across the entire world. The money flows through multiple intermediaries, each taking a small fee, and the whole thing relies on a global infrastructure that has been built over decades.

A closed-loop system is fundamentally different. As described by SDK.finance [SDK24], "closed-loop payment systems are financial arrangements that confine transactions to a specific network or ecosystem." In a closed-loop, the issuer and the acquirer are the same entity, which means a single operator controls the entire flow of money within a defined perimeter. Think of it like this: you load credits into the system, you spend those credits at authorized points, and whatever you did not spend, you can get back as a refund. The money never leaves the operator's ecosystem dur-

ing the transaction itself. The operator has full control over the rules, the fees, the authorization logic, and most importantly for our use case, the infrastructure.

There are several reasons why closed-loop systems are so attractive for festival environments, and the network situation is just one of them. The first and most obvious reason is the connectivity problem we discussed in Chapter 1. A festival is already a place where the network is going to have a bad time, so depending on external payment networks like Visa or Mastercard that need to reach authorization servers on the other side of the world is just asking for trouble. With a closed-loop system, all the authorization logic lives inside the operator's own infrastructure, so when the network has issues the system only needs to talk to itself.

The second reason is fees. Open-loop payment processing is not cheap. Every time someone pays with a credit or debit card, the merchant pays an interchange fee to the card issuer, a scheme fee to the card network, and a processing fee to the payment processor. For a small transaction like buying a beer at a festival, these fees can eat up a significant percentage of the sale. In a closed-loop system, the operator cuts out all these intermediaries. There is no interchange fee, no scheme fee, and no processor taking a cut of every transaction. The only point where external fees apply is when money enters the system through a top-up, which involves Stripe or another payment provider, but even then, almoast always these topup fees are lower due to negotiatiosn with the payment provider (that is most likely a strategic partner). Once the money is inside the closed loop, every subsequent transaction is free for the operator. For a festival processing tens of thousands of small transactions per day, this adds up to serious savings.

The third reason is perhaps the most interesting one, and it has to do with regulation. In the European Union, handling electronic money is a regulated activity that requires an e-money license under the Electronic Money Directive (EMD2) [Eur09]. Getting an e-money license is an expensive and time-consuming process that involves capital requirements, compliance obligations, and ongoing regulatory oversight. However, many festival payment systems operate in something of a regulatory gray area by using tokens or credits instead of displaying actual currency values. The idea is that when an attendee tops up their wristband or app, they are not depositing money into an electronic wallet in the regulatory sense. Instead, they are purchasing prepaid tokens that can be redeemed at participating vendors within the festival. This distinction matters because prepaid instruments issued for use within a limited network of service providers can fall under an exemption in the Payment Services Directive (PSD2) [Eur15], specifically the limited network exclusion. Under this exemption, if the payment instrument can only be used to acquire goods or services within a limited network of providers (like the vendors inside a festival), the operator may not need a full e-money or payment institution license. This is

also why many festival systems show balances in "tokens" or "credits" rather than in euros or dollars. Displaying the balance as "50 tokens" instead of "50 EUR" reinforces the narrative that these are not electronic money but rather prepaid vouchers for a specific set of services. Of course, this is a simplification and the legal situation varies by jurisdiction. Some regulators have taken a stricter view, and operators who process large volumes may still need to register or obtain a limited license. But the point is that the closed-loop, token-based model gives festival operators a much simpler regulatory path compared to running a full open-loop payment service.

The fourth reason is control. In a closed-loop system, the festival operator decides everything: transaction limits, refund policies, spending restrictions, which vendors are authorized, and how the settlement process works. There is no card scheme telling you what you can and cannot do. This level of control is particularly valuable for events where the operator needs to manage the entire economy, from how much people can top up to how refunds are handled after the festival ends. And the most important reason, of course, taking a percentage cut out of all the transactions inside the event without having to trust that vendors are honest, given that the only way to make a payment is through the operator's own system

Of course, there is one moment where the closed-loop system has to interact with the outside world, and that is when real money enters the system. When an attendee tops up their wallet, actual euros or dollars need to move from their bank account into the festival operator's account. This part requires an external payment provider like Stripe and an active internet connection. But once the money is inside the closed loop, all subsequent transactions are internal and can, in principle, be processed without any external dependency.

To understand where our system fits in the bigger picture, it helps to look at three existing categories of closed-loop payment systems that have dealt with similar challenges: transit cards, campus and event tokens, and the EMV offline authorization model.

2.1.1 Transit Card Systems

Transit card systems are probably the most successful example of closed-loop payments that work without constant connectivity. If you have ever used public transport in a major city, chances are you have used one. Examples include Suica in Tokyo, Oyster in London, or Octopus in Hong Kong. These systems have been around for decades, processing millions of transactions per day, and they offer some valuable lessons for anyone trying to build a payment system that needs to work fast and reliably.

The basic idea behind all of them is similar, at least in their original implementa-

tions. Shirakawa [Shi02] describes how Suica's FeliCa chip handles fare deduction locally at the gate through encrypted read-write operations, without needing a network round-trip. Chau and Poon [CP03] similarly describe Octopus as a "rechargeable contactless stored value smart card," meaning the balance lives on the chip itself. In both cases, the terminal communicates directly with the card, checks the balance, deducts the amount, and writes the updated balance back. FeliCa chips can complete this entire process in under 0.1 seconds. That said, these systems are not purely offline. Transaction logs are collected from terminals and uploaded to central servers periodically for reconciliation, fraud detection, and reporting. And newer versions, like the Suica 2.0 platform, have started moving fare calculation to a central server while still using the card chip for local read-write operations at the gate.

Suica is a good example to look at in detail. Launched by JR East in 2001, it was one of the first large-scale contactless payment systems in the world. Shirakawa [Shi02] describes how the system was designed to handle the rush hour crowds at Tokyo's train stations, where gates need to process passengers at a rate of about one person per second. The key to achieving this speed was keeping the authorization logic entirely local: the gate reads the card, checks the balance, deducts the fare, and opens. No network round-trip required. The card uses Sony's FeliCa chip [Son24b], a contactless IC with an embedded antenna that communicates using Manchester coding at 212 kbit/s in the 13.56 MHz range, requiring a proximity of 10 centimeters or less. It does not need a battery since it draws power from the reader when the card gets close enough, and the reader stops supplying power once the data transfer is done. Each transaction completes in approximately 0.1 seconds. Security is built into the protocol through mutual authentication: every time the card and reader communicate, a new encryption key is dynamically generated, which prevents fraud like impersonation or replay attacks. In 2011, Sony announced a new generation of FeliCa chips that adopted AES (Advanced Encryption Standard) encryption, and the newest generation announced in 2020 received the ISO/IEC 15408 EAL6+ certification, one of the highest security levels under the Common Criteria standard. Figure 2.1 illustrates how this mutual authentication works between the card and the gate reader.

The way mutual authentication works is basically a challenge-response protocol. The reader generates a random number and sends it to the card as a challenge. The card takes that number, encrypts it with the secret key that is stored inside the chip, and sends the encrypted result back. The reader already knows the key too, so it can check if the response is correct. If it is, the reader knows it is talking to a legitimate card and not a fake one. Then the process goes the other way around: the card sends its own random challenge to the reader, the reader encrypts it with the

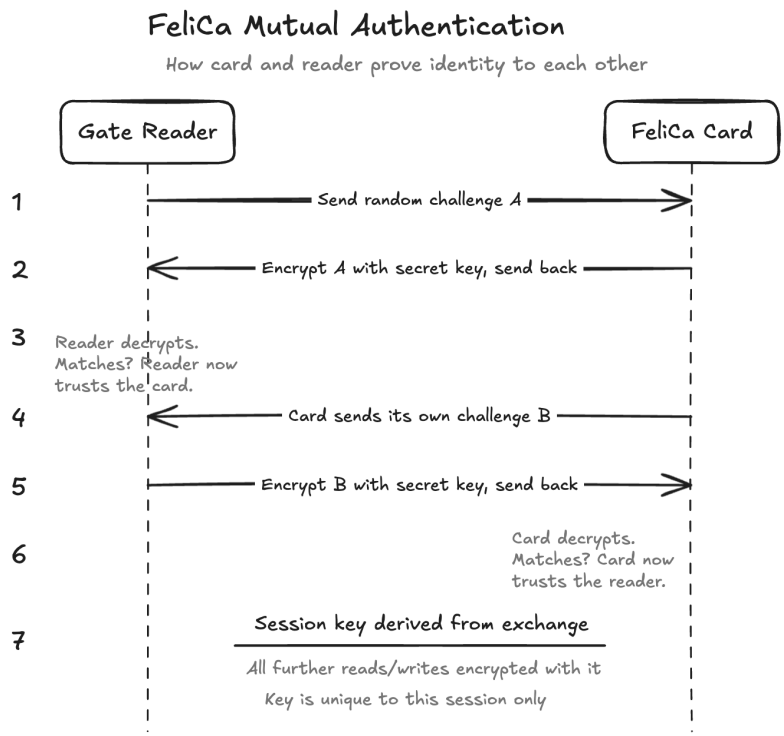

Figure 2.1: FeliCa mutual authentication flow between the gate reader and the contactless card. A new encryption key is derived for each session, making captured data useless for future transactions.

same key, and the card checks the response. Only after both sides have verified each other does the actual balance read and write happen. At the end of this exchange, both sides derive a session key that is used to encrypt everything that follows. This session key is unique to this one transaction and will never be reused.

This design also makes card cloning very hard to pull off. The secret key lives inside the chip's secure memory and it never leaves it. You cannot read it out, not even if you physically open the card. So even if someone has a reader and captures all the radio signals during a transaction, the encrypted responses they capture are useless for next time because the challenges will be different. To fake a valid response, an attacker would need the actual secret key, and there is no way to get it from the captured data alone. What about physical attacks, like putting the chip under a microscope and probing the circuits, or doing power analysis where you measure the chip's electricity consumption to try to figure out the key? That is exactly what the EAL6+ certification covers. To get that certification level, the chip has to be tested against these kinds of hardware attacks and prove it can resist them.

Octopus in Hong Kong follows a similar model but expanded the use case beyond transit. Launched in September 1997, it was actually the first contactless smart card system in the world to adopt Sony's FeliCa technology [Son24a], four years

before Suica. What started as a pure transit card evolved into a general-purpose stored-value card accepted at convenience stores, restaurants, parking garages, and even vending machines. By the early 2000s, it had over 11 million cards in circulation in a city of 7 million people, which gives you an idea of how widely adopted it became. The interesting thing about Octopus from our perspective is that it demonstrated that a closed-loop, card-stored-balance system could scale beyond its original domain while maintaining fast transaction times.

Now, what can we learn from these systems for our festival payment architecture? The first lesson is that speed matters a lot. Transit systems proved that sub-second transactions are not just nice to have, they are a hard requirement when you are processing large volumes of people who do not want to wait. At a festival, the same principle applies: nobody wants to stand in line for two minutes waiting for a payment to go through when they just want a beer.

The second lesson is about where the balance lives. Transit cards store the balance on the card chip, which is what makes offline operation possible. In a festival context, NFC bracelets can work in a similar way: the balance is written directly to the chip on the bracelet, and the vendor terminal reads and updates it locally, just like a Suica gate does with a FeliCa card. This means transactions can happen even without network connectivity. However, in our system we also support a smartphone app as an alternative to bracelets. With a phone, we cannot write balances directly to the NFC chip the way you can with a bracelet or a transit card. So for the app-based flow, the authoritative balance lives on the server, and during offline periods, vendor terminals work with cached snapshots of recent balances. This dual approach, bracelets with on-chip balances and phones with server-side balances, introduces its own set of challenges, which we will discuss in detail in Chapter 3.

The third lesson is about the limitations of the transit model. Transit systems operate in a very controlled environment: the terminals are fixed, maintained by professionals, connected to a private network, and the usage patterns are more or less predictable. A festival is a completely different story. Vendor terminals are mobile devices carried by staff who may not be very technical, the network is a temporary Wi-Fi setup in a field, and usage spikes happen unpredictably during meal breaks or between performances. So while transit systems show that offline closed-loop payments can work, their specific solutions do not transfer directly to our context.

2.1.2 Festival Payment Systems

Over the past decade, a whole industry has grown up around providing cashless payment solutions for festivals and events. Companies like Tappit, Intellitix, Weezevent,

and Glownet have deployed their systems at hundreds of events worldwide. The typical setup works like this: attendees receive an RFID wristband when they enter the festival. They load credits onto the wristband using cash top-up stations or an online portal before the event. At each vendor point, there is a handheld terminal that reads the wristband, deducts the purchase amount, and writes the new balance back to the chip. It is essentially the same model as transit cards, adapted for the festival context.

These systems have proven that cashless festivals are commercially viable. As mentioned in Chapter 1, Tappit reports that RFID systems reduce transaction times by up to 80 percent compared to cash and increase average spending by about 22 percent [Tap24]. The speed improvement comes from eliminating the need to count cash and make change. The spending increase is attributed to the psychological effect of not physically handing over money, which is consistent with the broader academic literature on cashless payment behavior [SBH24].

However, there are some significant limitations to the RFID wristband approach that motivated us to explore a different path. First, the RFID chips used in most festival wristbands have very limited storage and processing capability. They can store a balance and maybe a transaction counter, but they cannot run complex logic. This means that all the intelligence has to be in the terminal, and the terminal needs to handle things like transaction validation, balance updates, and security checks. If the terminal loses connectivity to the backend and its local cache is stale or incomplete, it has a problem.

Second, and this is the bigger issue, many of these RFID systems rely on MIFARE Classic chips, which have well-documented security vulnerabilities. Garcia et al. [GdKGM+08] demonstrated practical attacks against MIFARE Classic's proprietary CRYPTO1 cipher, showing that the encryption could be broken and the card contents cloned or modified. For a payment system, this is a serious problem because it means that a sufficiently motivated attacker could tamper with their balance. More modern chips like MIFARE DESFire have addressed these vulnerabilities, but they are more expensive, and many festival deployments still use the older technology to keep costs down.

Third, the architectures of these commercial systems are proprietary and essentially undocumented. There is no published academic analysis of how Tappit or Intellitix handle offline operation, conflict resolution, or balance reconciliation. This makes it hard to learn from their design decisions or to identify best practices. All we have are the occasional failure reports, like the Download Festival incident mentioned in Chapter 1, which tell us what went wrong but not how things were supposed to work.

Since there is not a lot of published research on how these systems work in prac-

tice, we talked to people who were involved in deploying payment infrastructure at Untold, one of the largest music festivals in Romania. From those conversations we learned that the typical approach is to deploy a local network on-site and bring dedicated servers to the venue, so that the core payment logic runs locally and does not depend on a remote cloud. Even with this setup though, the network is still unreliable enough that top-up operations can take two to four hours to synchronize across all terminals when the Wi-Fi coverage gets bad. This is a real problem for attendees who just loaded money and want to spend it immediately but the vendor terminal does not know about it yet. It shows that maintaining reliable Wi-Fi coverage is not just something nice to have, it is a core infrastructure requirement for any festival payment system. We discuss how our architecture handles these synchronization delays in Chapter 3.

Given these trade-offs, there are two main directions we could take. One option is to stick with the RFID wristband model and use NFC bracelets with on-chip balances, which gives us the advantage of fully offline transactions like transit cards but limits what we can do on the user side. The other option is to go with a smartphone app as the consumer device, where the balance lives on the server and payments happen via QR codes or NFC between the phone and the vendor terminal. A phone-based approach gives more flexibility for things like showing transaction history, doing top-ups from the app, and pushing notifications, but it means we depend more on the network since the balance is not stored locally on a chip. There is also a middle ground where we support both. We explore these options further in Chapter 3.

2.1.3 EMV and Offline Authorization

The EMV standard, named after its original creators Europay, Mastercard, and Visa, is the technology behind the chip cards that most people carry in their wallets. While EMV is an open-loop standard designed for the global payment network, it includes a mechanism for offline transaction authorization that is worth understanding because it addresses a similar problem to ours: how do you authorize a payment when you cannot reach the authorization server?

The way EMV offline authorization works is through a set of risk parameters stored on the card chip. The card knows its own balance limits, the maximum amount for a single offline transaction, the maximum number of offline transactions allowed before the card must go online, and the total cumulative amount that can be spent offline. When a merchant terminal cannot reach the bank's authorization server, it can still approve transactions by checking these parameters locally. If the transaction falls within the allowed limits, it gets approved offline. If it ex-

ceeds them, the terminal declines the transaction or forces an online authorization attempt.

This is a clever approach because it distributes the risk. The card issuer decides how much offline risk they are willing to accept for each cardholder, and those limits are baked into the card at the time of issuance. A premium cardholder might have higher offline limits than a basic cardholder. The merchant also has some say in this: their terminal can have its own floor limit below which it will always accept transactions without going online.

However, there is a well-known problem with EMV offline authorization: the system trusts the card to honestly report its own state. Murdoch et al. [MDAB10] demonstrated a man-in-the-middle attack on the EMV protocol where a device placed between the card and the terminal could trick the terminal into believing that a PIN had been correctly verified when it had not. While this specific attack targets the PIN verification rather than the offline balance logic, it illustrates a broader point: any system where the authorization decision depends on data stored on a device in the user's possession is vulnerable to tampering if the security of that device is compromised.

For our system, the EMV offline model offers both inspiration and a warning. On the inspiration side, the idea of using spending limits to control offline risk is directly applicable. We implement something similar: during offline periods, vendor terminals enforce a maximum transaction amount and a maximum total offline spend per consumer, which limits the potential damage from double-spending or stale balance snapshots. On the warning side, the EMV experience shows that offline authorization always involves a trade-off between availability and security. The more you allow offline, the more risk you accept. Our system handles this by keeping offline periods as short as possible and treating offline mode as a temporary fallback rather than a primary operating mode.

2.2 Offline-First Architecture Patterns

When you build a system that needs to work without a network connection, you are basically deciding where on a spectrum your application sits. On one end you have fully online systems, where every operation needs a round-trip to a server and nothing works without internet. On the other end you have fully offline systems, like the FeliCa transit cards we discussed earlier, where everything happens locally and the network is only used for background synchronization. Most real-world applications fall somewhere in between.

For our festival payment system, we need to be somewhere in the middle of this spectrum. The system should work online by default, because that gives us the

strongest guarantees about balance correctness and transaction ordering. But when the network goes down, and at a festival it will go down, the system needs to keep processing payments using whatever data it has cached locally. This means we need architectural patterns that support both modes of operation and can switch between them without losing data or creating inconsistencies that cannot be resolved later.

In this section we look at four architectural patterns that are relevant to this problem. Each one addresses a different aspect of building systems that can work without constant connectivity.

2.2.1 Local-First Software

The term "local-first software" was introduced by Kleppmann et al. [KWvHM19] in a 2019 paper that argued for a different way of thinking about where data lives in an application. The basic idea is that the primary copy of the data should be on the user's local device, not on a remote server. The cloud still exists, but its role is to sync data between devices rather than to be the single source of truth. If the server goes down, or the network disappears, the application keeps working because the data is right there on the device.

Kleppmann and his co-authors defined seven ideals for local-first software: no spinners (the app should be fast because it reads from local storage), the user's work should be available on multiple devices, the network should be optional, collaboration should be seamless, the data should be accessible for a long time (not locked behind a service that might shut down), security and privacy should be built in, and the user should have full ownership and control over their data.

Now, our festival payment system does not fit neatly into the local-first model, and there is a good reason for that. In a collaborative document editor, if two people edit the same paragraph and the results get merged a bit awkwardly, that is annoying but not catastrophic. In a payment system, if two terminals both think a user has 50 euros and each one authorizes a 40 euro purchase, you have just lost 30 euros. Money is a conserved quantity. You cannot merge two versions of a balance the way you merge two versions of a text document.

That said, some of the local-first principles are directly useful for us. The idea of keeping a local copy of the data so the app does not freeze when the network drops is exactly what our vendor terminals need to do. The difference is that in our case, the local copy is a read cache with limited write authority, not the primary copy. The server remains the source of truth, and offline transactions are treated as provisional until they get confirmed by the server. This is a weaker version of local-first, but it is the right trade-off for a system where financial correctness matters more than collaboration flexibility.

2.2.2 Conflict-Free Replicated Data Types (CRDTs)

CRDTs, or Conflict-Free Replicated Data Types, are data structures specifically designed for systems where multiple nodes can modify the same data independently, without coordinating with each other, and still end up in a consistent state. They were formalized by Shapiro et al. [SPBZ11] in 2011, and the key property that makes them work is that all operations on the data structure are designed to be commutative, associative, and idempotent. What this means in practice is that it does not matter what order the updates arrive in, or if some updates arrive more than once. The result will always be the same.

The simplest example is a G-Counter (grow-only counter). Imagine you have five terminals at a festival, and each one counts how many transactions it has processed. Each terminal only increments its own counter. When they sync, you just take the maximum value for each terminal's counter and sum them up. No conflicts possible. A PN-Counter extends this idea by having two G-Counters, one for increments and one for decrements, and the actual value is the difference between the two.

This sounds like it could work for tracking balances, but there is a catch. A balance is not just a counter that goes up and down freely. Money is conserved: you cannot spend more than you have. CRDTs are great at ensuring convergence (everyone ends up with the same value), but they do not enforce invariants like "the balance must not go below zero." If two terminals independently authorize purchases against the same balance while offline, a naive CRDT approach would let both go through, even if the total exceeds the available balance. You would only discover the overspend after synchronization.

Kleppmann [KB17] explored this limitation in the context of JSON CRDTs and noted that while CRDTs guarantee convergence, they do not guarantee that the converged state is one that any individual node would have accepted. For payment systems, this is a real problem. We cannot just let every offline transaction go through and hope the math works out later.

So for a payment system, pure CRDTs are not enough on their own. You need something extra on top to deal with the fact that balances have constraints. There are a few ways to handle this. One option is to record offline transactions as events and have the server replay and validate them when connectivity returns, rejecting any that would overdraw the balance. Another option is to pre-allocate spending limits per terminal so that each terminal can only authorize up to a fixed amount while offline, which reduces the chance of overspending in the first place. A third option is some combination of the two. We explore these trade-offs further in Chapter 3.

2.2.3 Event Sourcing and CQRS

Event sourcing is a pattern where instead of storing the current state of an object (like "balance = 47.50"), you store the entire sequence of events that led to that state ("topped up 50.00, bought beer for 2.50"). The current state can always be reconstructed by replaying the events from the beginning. Fowler [Fow05] describes event sourcing as a way to "ensure that all changes to application state are stored as a sequence of events," and one of the main benefits is that you get a complete audit trail for free.

This pattern fits really well with offline payment processing. When a vendor terminal goes offline, it cannot update the central database directly. But it can keep recording events locally: "user X paid 5.00 for item Y at timestamp Z." Each event is self-contained and immutable. When the terminal reconnects, it uploads its event log to the server, and the server can process these events in order to update the authoritative state. If there is a conflict (say, the user ran out of money between two offline purchases), the server can detect it during replay and handle it according to whatever policy the operator has defined.

CQRS, which stands for Command Query Responsibility Segregation, is a related pattern that is often used together with event sourcing. The idea, as described by Young [You10], is to separate the write side of the system (commands that modify state) from the read side (queries that return data). In a traditional system, the same data model handles both reads and writes. In CQRS, you have one model optimized for processing writes (accepting payment events) and another model optimized for fast reads (showing the current balance on a screen).

For our system, this separation is useful because the write side and the read side have very different requirements. On the write side, we need strong ordering guarantees and conflict detection. On the read side, vendor terminals need to quickly look up a user's balance to decide whether to authorize a payment, and this lookup needs to work even when the terminal is offline. By separating these concerns, we can have the read model work from a local cache while the write model queues events for later synchronization, without the two getting in each other's way.

2.2.4 Edge Computing

Edge computing is the idea of moving computation closer to where the data is generated, instead of sending everything to a central server or cloud for processing. Shi et al. [SCZ+16] describe it as "pushing the frontier of computing applications, data, and services away from centralized nodes to the logical extremes of a network." In practice, this means doing as much work as possible on the device that is closest to the user, and only involving the cloud when necessary.

In the context of our festival payment system, the vendor terminals are the edge nodes. Each terminal runs a local instance of the payment logic: it can read a user's balance from its local cache, validate a transaction against the cached data, record the transaction as an event, and update its local view of the balance. All of this happens on the terminal itself without any network communication. The cloud server (or the on-site local server, as we discussed in the context of the Untold festival) acts as the central coordinator that collects events from all the edge nodes, resolves conflicts, and pushes updated state back out.

This works well for festival environments because the network is the weakest link in the whole system. The less you depend on it for individual transactions, the better. If the terminal can handle a payment entirely on its own, then a network outage does not mean a payment outage. It just means that the synchronization of data gets delayed. The trade-off, of course, is that edge nodes are working with potentially stale data. A terminal that has been offline for 30 minutes might not know about a top-up that happened 20 minutes ago. How we handle this staleness, and what limits we put on edge-processed transactions to manage the risk, is something we cover in detail in Chapter 3.

2.3 Conflict Resolution in Distributed Systems

When multiple vendor terminals at a festival are processing payments independently, without being able to talk to each other or to a central server, they are going to end up with different views of the world. Terminal A might think a user has 50 euros because it processed a top-up, while Terminal B still shows the old balance of 20 euros because it has not received the update yet. If both terminals authorize purchases during this window, the system will have conflicting records that need to be sorted out once everyone reconnects.

This is not a new problem. Distributed systems have been dealing with conflicting state for decades, and there is a solid body of theory and practice around how to handle it. In this section we go through the main concepts that are relevant to our case: what trade-offs are fundamental and cannot be avoided, how systems achieve consistency without requiring all nodes to agree in real-time, how to figure out the order of events when there is no shared clock, and how to let nodes work independently with the plan of fixing things up later.

2.3.1 The CAP Theorem

In 2000, Eric Brewer presented a conjecture at the ACM Symposium on Principles of Distributed Computing that would become one of the most cited results in dis-

tributed systems. The conjecture, which Gilbert and Lynch [GL02] formally proved in 2002, says that a distributed system can only guarantee two out of three properties at the same time: Consistency (every node sees the same data at the same time), Availability (every request to a non-failing node gets a response), and Partition Tolerance (the system keeps working even when the network between nodes is broken).

The important word here is "partition tolerance." In a festival environment, network partitions are not an edge case, they are a regular occurrence. When the Wi-Fi drops between two parts of the venue, the vendor terminals on each side are partitioned from each other and from the server. At that point, according to CAP, we have to choose: do we want consistency or availability?

If we choose consistency, then when a terminal cannot reach the server to verify a balance, it should refuse the transaction. The data stays consistent, but the system is unavailable and the attendee cannot buy anything. If we choose availability, the terminal goes ahead and authorizes the payment based on its local cache, even though it cannot be sure the data is up to date. The payment goes through, the attendee is happy, but we might end up with inconsistencies that have to be fixed later.

For a festival payment system, refusing transactions because the network is down is not really an option. Nobody is going to accept a system that stops working every time the Wi-Fi gets flaky, which at a festival is all the time. So during network partitions, our system leans toward availability. Transactions keep going through, and consistency is restored after the partition heals. This is not a unique choice. Many real-world systems choose availability during partitions and then run a reconciliation process afterward. That is essentially what we are doing.

2.3.2 Eventual Consistency

If we are giving up strong consistency during network partitions, the question becomes: what kind of consistency can we actually promise? The answer, for most distributed systems that prioritize availability, is eventual consistency. Vogels [Vog09] defines it simply: if no new updates are made to a piece of data, then eventually all replicas will converge to the same value.

The word "eventually" is doing a lot of work in that definition. It does not say when the replicas will agree, just that they will at some point. In practice, this usually happens pretty fast once connectivity is restored. But during the partition, different nodes can have different views of the data, and that is considered acceptable.

The most famous real-world example of eventual consistency is Amazon's Dynamo, described by DeCandia et al. [DHJ+07]. Dynamo is a key-value store that Amazon built for their shopping cart and other services that need to be available

all the time, even during failures. Dynamo allows writes to happen on any node, even if that node cannot reach the others. When the partition heals, Dynamo uses application-specific logic to merge conflicting versions.

For our system, eventual consistency means the following: while terminals are offline, they work with their local data and record all transactions locally. When they reconnect, they send their transaction logs to the server. The server processes everything, figures out the correct state of each balance, and pushes the updated data back to all terminals. At that point, everyone is consistent again. The window during which terminals might have stale or conflicting data is the offline period, and the goal of the architecture is to make this window as short as possible and to limit the damage that can happen during it (for example, by capping how much a terminal can authorize while offline).

2.3.3 Transaction Ordering and Why Timestamps Are Enough

One of the classic problems in distributed systems is figuring out the order in which things happened across different nodes. The standard solutions from the literature are logical clocks and vector clocks, which use counters instead of real time to establish ordering. These are necessary in systems like high-frequency trading platforms, where events happen microseconds apart on different machines and physical clock drift is larger than the interval between events.

However, for a festival payment system, the situation is fundamentally different. Transactions happen at human speed. A person walks to a vendor, waits in line, orders something, taps their wristband. The gap between one transaction and the next, even for the same user, is measured in minutes, not microseconds. Even cheap NTP synchronization keeps clocks within a few hundred milliseconds of each other, and that kind of drift is completely irrelevant when the events we care about are minutes apart.

This means that plain wall-clock timestamps, recorded by each vendor terminal at the moment of the transaction, give us unambiguous ordering in practice. There is no realistic scenario where two transactions from the same user happen so close together that clock drift changes the order. The physical constraint of the wristband model reinforces this: a user can only be at one vendor at a time, so transactions from the same user are naturally serialized by their physical presence.

The practical advantage of using timestamps over vector clocks is significant. With timestamps, vendor terminals do not need to communicate with each other at all during offline periods. Each terminal just records its local time with each transaction. When the network comes back, everyone pushes their logs to the server, the server sorts by timestamp, and we are done. There is no vector state to main-

tain, no counter synchronization between devices, and no extra complexity in the protocol. For a system that needs to work reliably in a festival environment where connectivity is unreliable, this simplicity is a real advantage.

Chapter 3

System Architecture

This chapter goes through the architecture of the closed-loop payment system. We start with the big picture, explain why we chose this approach, and then go into each component. The idea is to keep things simple: one server, one app for everyone, and reliable network whenever possible. If the network goes down, the balance cached on the bracelet is what decides whether a payment is approved.

3.1 High-level overview

Most existing festival payment systems need a technical team on-site to set up local servers and a private network for the duration of the event. As discussed in Section 2.1.2, festivals like Untold bring dedicated servers to the venue because the balance lives on a remote server and every transaction needs a network round-trip.

Our system takes a different path. We aim to always have reliable internet using Starlink as the primary uplink, paired with a redundant network topology. The wristband-as-cache fallback exists for residual edge cases, but the goal is to never need it. This means there is nothing to deploy at the venue: no local servers, no private network, no on-site IT team. The organizer creates an event from the admin dashboard, configures it, approves vendors, and distributes wristbands. The server runs in the cloud the same way it runs for every other festival on the platform.

The system has four main components: a backend server, a mobile application for attendees and vendors, a web-based admin dashboard, and NFC wristbands that carry the token balance for each attendee. The server is a single multi-tenant service that manages all festivals from one deployment. The mobile app is what attendees and vendors interact with: attendees see their balance and history, vendors use it as a soft point-of-sale terminal. Admins manage the event through a separate web dashboard. The wristband is where the balance actually lives during the event.

3.1.1 The hybrid reconciliation model

The core architectural decision is how the wristband and the server share responsibility for the balance.

The state is split across the two sides. Each wristband stores three fields per event: `balance`, `debit_counter` (incremented on every chip-side payment), and `credit_counter_seen` (the highest server credit sequence the chip has acknowledged). The server keeps the mirror image: `balance`, `debit_counter_seen`, and `credit_counter`. Balances are event-scoped, so the same person attending two festivals has two independent wallets.

For this thesis demo we use a **plain NTAG-class NFC chip** without a secure element. The hybrid model does not depend on chip-side cryptography: it only needs the chip to store a few fields, increment a counter atomically, and keep what was last written to it. The security implications and the upgrade path to authenticated chips are discussed in Chapter 5.

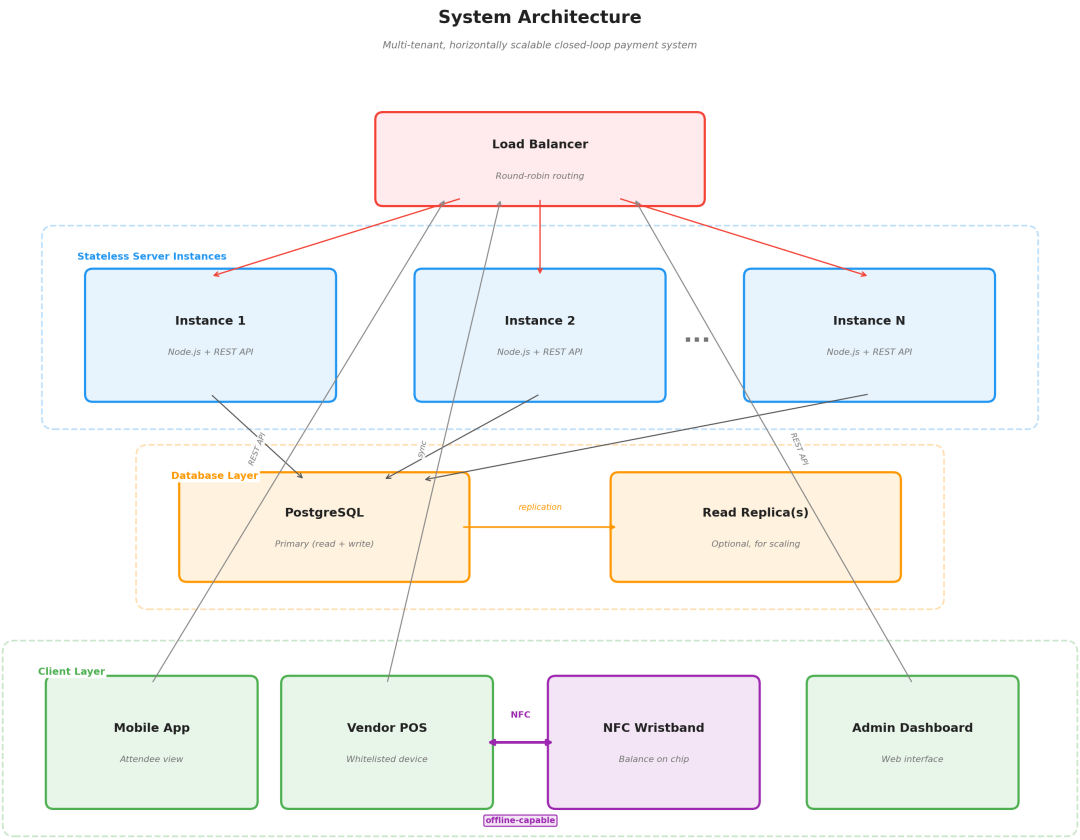

Figure 3.1: High-level system architecture. Debits happen directly between the vendor device and the NFC wristband. Credits go through the backend, which is also responsible for top-ups, sync, and reporting.

3.2 Balance reconciliation model

A closed-loop payment system with offline-capable terminals has two writers that can mutate the balance: the server (top-ups) and the chip (vendor payments). During a network partition both can act on their own, and once connectivity is back the system has to converge on a correct final value. Our hybrid model handles this by keeping the two writers in separate lanes that never overlap.

3.2.1 State on each side

The wristband stores three reconciliation fields per event, plus identity fields written at provisioning:

- `balance`: cached balance in minor units, what the chip can spend from when offline.
- `debit_counter`: monotonic counter, incremented by one on every chip-side debit. The chip is the only writer.
- `credit_counter_seen`: the highest server-side credit sequence number the chip has acknowledged.
- identity fields `bracelet_uid` and `event_id`, written once at provisioning.

The server stores the mirror image, per wristband per event:

- `balance`: the canonical balance.
- `credit_counter`: monotonic counter, incremented by one on every server-side credit. The server is the only writer.
- `debit_counter_seen`: the highest `debit_counter` value the server has reconciled from any terminal.

The two counters are independent monotonic sequences, each with one writer. Because each lane has exactly one writer, there is no merge conflict to resolve within a lane.

3.2.2 Invariants

The correctness of the model rests on three invariants:

1. **The chip never increases its own balance.** Every chip-side mutation is a debit: `balance -= amount` together with `debit_counter += 1`, applied as one atomic write.

2. **The server is the only source of credits.** Top-ups, refunds, and operator adjustments mutate the server's balance and increment `credit_counter`. The chip picks up these credits passively on the next tap by an online terminal.

3. **Counters are monotonic with a single owner each.** `debit_counter` only ever grows on the chip; `credit_counter` only ever grows on the server.

As long as these three hold, reconciliation reduces to taking the union of two non-conflicting streams.

3.2.3 The four operations

Top-up. Always online. The server checks the payment, increments `balance` and `credit_counter`, writes a transaction row, and acknowledges. The chip is not touched and gets reconciled on the next tap.

Online debit. The terminal reads the chip and forwards the request to the server with the chip's current `debit_counter` and `credit_counter_seen`. The server applies any pending credits, validates the debit, and returns the new triple `(balance, debit_counter, credit_counter_seen)`, which the terminal writes atomically to the chip.

Offline debit. The terminal reads the chip, checks the balance, and performs an atomic decrement on the chip. It generates an idempotency key and queues the transaction locally, draining the queue in the background as connectivity allows.

Sync. When a terminal regains connectivity, it pushes queued transactions to the server in batches. The server applies them idempotently using the per-transaction key, updates `balance`, and advances `debit_counter_seen` to the highest counter value observed.

3.2.4 Worked example

Consider a top-up and an offline debit happening at the same time. Starting state: chip `(balance=50, debit_counter=0, credit_counter_seen=0)` and server `(balance=50, credit_counter=0, debit_counter_seen=0)`.

The user pays 10 tokens at an offline terminal. The chip becomes `(40, 1, 0)` and the terminal queues a debit of 10. In parallel, a 20-token top-up goes through the server, which becomes `(70, 1, 0)`.

The user then taps an online terminal. The terminal reads `(40, 1, 0)` and forwards it to the server. The server sees `credit_counter=1 > credit_counter_seen=0`, applies the pending 20-token credit, and writes `(60, 1, 1)` to the chip.

When the offline terminal eventually syncs the queued debit, the server applies it: balance goes from 70 to 60, and `debit_counter_seen` advances from 0 to 1.

Both sides converge at `balance=60` with no conflict resolution because the two streams never collided.

3.2.5 Lost transactions

The model does not magically prevent loss. If a terminal dies before draining its queue, the vendor served the customer and the chip subtracted the amount, but the server has no record. The gap is detectable: when `chip.debit_counter` exceeds `debit_counter_seen` plus the count of pending queued transactions, the difference represents lost debits.

The policy is to trust the chip's balance immediately, wait a configurable grace period for late syncs, and after that reconcile by writing the chip's balance to the server and logging the discrepancy. The festival absorbs the loss for those transactions; vendors with local receipts can claim manually. The exposure is bounded by the sync interval, an operational risk rather than a correctness bug.

3.3 Lessons from related systems and patterns

The hybrid model mixes ideas from transit cards, EMV, and the distributed systems literature.

Transit card systems like Suica and Octopus (Section 2.1.1) prove that an on-chip balance works at scale: the balance lives on the chip and the gate does not call any server. Our wristband borrows this core idea, with the same split as Suica (chip-only on spending, back-office on credit).

From commercial festival systems like Tappit and Intellitix (Section 2.1.2), we mainly learned what to avoid: requiring an IT team and on-site servers. Conversations about the Untold deployment confirmed that even with on-site infrastructure, top-up sync delays of two to four hours were common when Wi-Fi got bad. Our architecture sidesteps this entirely.

The EMV offline authorization model (Section 2.1.3) gave us spending limits as a risk control. The event admin could hypotheticly configure maximum transaction amounts and a maximum total offline spend per wristband, as a safety net for the case where someone tampers with a chip, but this should not be doable with proper secured chips.

From the architectural patterns covered in Section 2.2, the relevant takeaways are: **local-first software** [KWvHM19] does not fit payment systems because conflicting balance updates cannot just be merged; **CRDTs** [SPBZ11] are overkill because lane separation already solves the convergence problem with two integers; **event sourcing** is the one pattern we use directly, with every transaction recorded as an

immutable event for audit and settlement; and **edge computing** shows up at two levels, the vendor phones running debit logic locally, and the server deployed in a region close to the festivals it serves.

3.3.1 Offline operation

During online operation, vendor terminals send each transaction to the server right after it is recorded locally. When the network goes down, the wristband interaction does not change at all. Transactions accumulate in the local queue on the vendor device instead of being sent immediately. Once connectivity returns, the device pushes its queued transactions in a batch. The full convergence mechanics are covered in Section 3.2.

3.3.2 Top-ups: the one thing that needs the network

Topping up is the one operation that cannot happen offline. Real money moves from the attendee's bank account through Stripe into the operator's account, which requires an internet connection. Restricting top-ups to online-only is also what keeps the credit lane fully under server control.

Top-ups happen in two ways. Attendees can top up through the app using Stripe; once the payment clears, the server records the new balance and increments `credit_counter`, and the next vendor tap picks up the pending credit. Cash top-up stations at the event submit the credit through the server, which then writes the new balance to the wristband on the spot.

3.4 The mobile application

There is one mobile app, built in React Native, that attendees and vendors both download. What the user sees depends on their role: attendees see their balance, vendors get a POS terminal. We went with React Native because the same codebase needs to run on regular phones (attendees) and on devices used as POS terminals (vendors) that require NFC support.

3.4.1 Attendee view

Attendees see a simple screen showing their token balance, a transaction history, and a way to top up using Stripe. The balance shown in the app is the last known state synced from the server; the real balance during the event lives on the wristband. The app is mostly informational for attendees.

3.4.2 Vendor POS mode

When an admin has whitelisted a device, the app switches to POS mode on that device. The vendor enters the amount, the customer taps their wristband, the app reads the balance, checks it, deducts the amount, writes the new balance back, and logs the transaction. If online, the transaction is sent to the server immediately; otherwise it goes into the local queue.

To become a vendor, the business provides basic information (business name, contact, products). The admin reviews and approves the application, then whitelists specific devices (by device ID) as POS terminals. This whitelisting matters because not just any phone can start accepting payments.

3.5 Admin dashboard

The admin dashboard is a separate web application built in React. Making it a website means any organizer can access it from any browser without downloading anything. The dashboard covers four areas:

Event setup. Create the event, set the token-to-currency rate, define spending limits, configure vendor commissions, and set dates. The event has a lifecycle (setup, active, settlement, closed) and the dashboard controls the transitions.

Vendor management. Review vendor applications, approve or reject them

Monitoring and statistics. Real-time transaction volume and other stats

3.6 Backend server

The backend is built with NestJS, which enforces a clean separation between controllers, services, and repositories, and gives us guards, interceptors, and dependency injection out of the box.

The server is deployed on DigitalOcean, with a serverless Neon PostgreSQL database in the same region. We deploy to a European region to keep latency low for festivals in Romania and the rest of Europe.

The REST API covers four areas: authentication and accounts, event management, vendor management, and sync/reconciliation.

3.6.1 Multi-tenancy

The server handles all festivals from a single deployment. Every table is scoped by an `event_id` column, which acts as the tenant boundary. API calls include the

event context (path parameter or token claim), and queries are filtered accordingly. Adding a new festival is just creating a row in the events table.

3.6.2 Horizontal scalability

The server is stateless: no session data or cached state in memory. Every request gets what it needs from the database and from the JWT token. We can run as many instances as we want behind a load balancer with no sticky sessions or inter-instance communication. For most festivals a single instance is enough, since most transactions happen on the wristband without touching the server.

3.7 Database: PostgreSQL

The database is PostgreSQL, running as a single instance with the option to add read replicas later. The schema follows the hybrid model directly:

- `users`, `sessions`, `accounts`, `verifications`: identity and auth, managed by Better Auth.
- `events`: per-event configuration (organizer, status, currency, token rate, dates, location).
- `event_members`, `vendor_members`: per-event and per-vendor roles. Authority is held here, not on the global `users.role` column.
- `vendors`: vendor application per event, with status (`pending`, `approved`, `rejected`, `suspended`) and commission rate.
- `event_bracelets`: the wallet, one row per (event, wristband). Holds the canonical `balance`, `debit_counter_seen`, and `credit_counter`, plus the active token (`tokenIssuedAt`, `tokenExpiresAt`, `tokenVersion`) and the assignment status (`active`, `revoked`, `replaced`). A partial unique index keeps at most one active assignment per (event, wristband) and per (event, user).
- `event_tickets`: admin-issued QR invitations that bind a wristband to a user at the gate.
- `transactions`: one row per money movement, scoped to a bracelet via `event_bracelet_id`. `type` is `debit` or `credit`; `debit_counter` or `credit_counter` is filled in accordingly. The `offline` flag and `idempotency_key` make sync batches safe to retry.

- `payment_intents`: provider-agnostic top-up records. On webhook success the credit lands on the exact bracelet referenced by the intent.
- `audit_logs`: append-only trail of admin actions, indexed by event, user, and entity.

Every table that belongs to an event is reachable from the event row, either directly through an `event_id` foreign key or through `event_bracelets`, which gives us the multi-tenant boundary without a tenant column on every table.

A common worry with using one database for everything is whether it can handle the load. In January 2026, OpenAI published a blog post [Ope26] describing how they run ChatGPT, with over 800 million users, on a single primary PostgreSQL on Azure plus around 50 read replicas, processing over a million queries per second at low double-digit millisecond p99 latency. If that setup serves 800 million users, a festival with 50,000 attendees is not what breaks PostgreSQL. Our load is also lighter than a typical web app because most payments happen on the wristband and never touch the database in real time.

If the platform grows to the point where the database becomes a bottleneck, we can follow the same playbook: read replicas, connection pooling (PgBouncer), and table partitioning, all standard techniques that do not require changing application code.

Chapter 4

Implementation

This chapter describes how the architecture from Chapter 3 was turned into running code. The focus is on the parts that are specific to the hybrid balance model: the database schema rework, the on-chip data layout, the sync endpoint, and the reconciliation logic that ties everything together. Topics like screen design, navigation, or generic CRUD endpoints are mentioned only briefly because they do not say much about the system as a whole.

4.1 Implementation overview

The system is built on three codebases that live in the same monorepo. The backend is a NestJS server in `backend/`, talking to a Postgres database through Drizzle ORM. The vendor terminal is a React Native application built with Expo, in `softpos/`, using `react-native-nfc-manager` for chip reads and writes. The thesis itself lives in `docs/`.

The whole stack uses TypeScript end to end. This was a deliberate choice. The data shapes that travel between the chip, the terminal, and the server are easy to get wrong, and a single set of types shared between client and server catches most of those mistakes at build time instead of in a tent at three in the morning. Drizzle's schema definitions also act as the source of truth for the database, so the table layout and the application types stay in sync without a separate code generation step.

Postgres was picked for the database because the workload is mostly small writes against a single bracelet row at a time, with occasional reporting queries on top. Serializable isolation, which the reconciliation algorithm relies on, is well supported and behaves predictably. The terminal stores its offline queue in AsyncStorage, which is enough for the volumes a single vendor handles between sync windows.

4.2 Database schema for the hybrid model

The hybrid model from Chapter 3 required a real cleanup of the schema. The original design had a generic `wallets` table where each user owned a single balance, shared across events. That made sense when the system was thought of as a closed-loop wallet platform, but it does not match how festivals actually work. An attendee at one festival does not expect leftover credit to appear at a different festival run by a different organizer, and an organizer wants their event balances to be isolated for accounting and refunds. Balance is therefore event-scoped, and the natural place for it is the row that already represents the link between an attendee and a specific event.

That row is `event_bracelets`, and it now carries the balance directly. Three new columns were added: `balance` (an integer count of cents, defaulting to zero), `debit_counter_seen` (the highest `debit_counter` value the server has applied from the chip for this bracelet), and `credit_counter` (a server-owned counter that increases by one every time the server credits the bracelet, for example after a successful top-up). The two counters are independent on purpose. Debits are allocated by the chip when a payment happens at a vendor, and credits are allocated by the server when money flows in. Keeping them separate means neither side has to coordinate with the other to pick a fresh value.

The `transactions` table was reshaped to match. The legacy `transaction_counter` column was dropped because it tried to merge debits and credits into one sequence, which made it impossible to tell which side a row was authored by. In its place, two nullable columns were added: `debit_counter`, set on debit rows, and `credit_counter`, set on credit rows. Exactly one of the two is populated for any given transaction. The foreign key `wallet_id` was retargeted to `event_bracelet_id`, and the redundant `event_id` column was removed because the event is reachable through the bracelet. The same retargeting was applied to `payment_intents.wallet_id`.

Two tables were dropped entirely. The `wallets` table is no longer needed because the balance moved to `event_bracelets`, and `vendor_invitations` was removed because the invitation flow was replaced by a simpler model where adding a user as a vendor creates the user account on the spot if they do not already have one. The remaining membership tables (`event_tickets`, `event_members`, `vendor_members`) were kept as is.

The end result is a schema that maps cleanly onto how operators think about a festival. A bracelet belongs to an event, the balance lives on that bracelet, and every movement of money against it is logged in `transactions` with a counter that says whether it was a debit (allocated by the chip) or a credit (allocated by the server).

Table	Column	Purpose
event_bracelets	balance	Current balance in cents, event scoped
event_bracelets	debit_counter_seen	Last debit counter applied from chip
event_bracelets	credit_counter	Server-owned credit counter
transactions	debit_counter	Set on debit rows, nullable
transactions	credit_counter	Set on credit rows, nullable
transactions	event_bracelet_id	Replaces wallet_id
payment_intents	event_bracelet_id	Replaces wallet_id

Table 4.1: New and changed columns introduced by the hybrid model rework.

4.3 The reconciliation service and sync endpoint

The sync endpoint is the single place where the chip state, the terminal's offline queue, and the server ledger meet. It is exposed by the backend as a single POST per bracelet, and it is intentionally chunky rather than chatty: the terminal sends everything it has for one bracelet, and the server returns everything the terminal needs to write back to the chip and to clean up its local queue.

```
POST /api/v1/events/:eventId/wristbands/:wristbandUid/sync
Authorization: Bearer <token>
Content-Type: application/json

Request:
{
  "chipState": {
    "balance": <int cents>,
    "debit_counter": <int>,
    "credit_counter_seen": <int>
  },
  "pendingDebits": [
    {
      "idempotencyKey": "<uuid>",
      "amount": <int cents>,
      "vendorId": "<uuid>",
      "counterValue": <int>,
      "deviceId": "<string>",
      "clientTimestamp": "<ISO 8601>"
    }
  ]
}
```

```
Response 200:
{
  "serverState": {
    "balance": <int cents>,
    "debit_counter_seen": <int>,
    "credit_counter": <int>
  },
  "applied":  ["<idempotencyKey>", ...],
  "rejected": [
    { "idempotencyKey": "...",
      "reason": "duplicate" | "insufficient_funds"
              | "counter_gap" | "invalid" }
  ],
  "chipShouldWrite": {
    "balance": <int cents>,
    "credit_counter": <int>
  }
}
```

The terminal does not send the chip's HMAC, because by the time `pendingDebits` reaches the server those debits have already been written to the chip and the HMAC has been recomputed. What matters at sync time is whether each entry in the queue has actually landed on the server's ledger.

The reconciliation algorithm runs inside a serializable Postgres transaction that locks the bracelet row for the duration of the call. Serializable isolation is important here: two terminals that both held the same bracelet during overlapping offline windows can hit the sync endpoint at the same time, and the database needs to make their effects look like a strict ordering. With a weaker isolation level, a race between two reconciliations could let the same chip counter be applied twice, or could allow the balance to go negative.

```
1. Open a serializable transaction and lock the bracelet row.
2. Sort pendingDebits by counterValue ascending.
3. For each pending debit:
     a. If idempotencyKey already exists in transactions,
        record it as duplicate and continue.
     b. If counterValue <= bracelet.debit_counter_seen,
        record it as duplicate and continue.
     c. If bracelet.balance < amount,
        record it as insufficient_funds and continue.
```

```
     d. Decrement bracelet.balance by amount.
     e. Set bracelet.debit_counter_seen = counterValue.
     f. Insert a transaction row with debit_counter
        = counterValue and the supplied idempotencyKey.
4. Read the current bracelet.balance and bracelet.credit_counter.
5. Build chipShouldWrite from that current state.
6. Commit the transaction.
7. Return serverState, applied, rejected, and chipShouldWrite.
```

The four rejection reasons cover the cases that can plausibly happen in the field. The `duplicate` reason is returned when the same debit reaches the server twice, either because the terminal retried after a flaky response or because two terminals scanned the same chip at different points and reported the same counter. Idempotency on `idempotencyKey` catches the first case, the counter check catches the second. The `insufficient_funds` reason fires when the chip view of the world disagrees with the server view, usually after a top-up that has not yet been written back to the chip; the debit was authorized offline based on a stale chip balance, and on reconciliation the server cannot honour it. The `counter_gap` reason is reserved for the case where the server expected a contiguous run of debit counters and the queue has a hole, which suggests a debit went missing or was tampered with. The `invalid` reason is the catch-all for malformed entries.

The response carries `chipShouldWrite`, which is the authoritative state the terminal must push to the chip before completing the sync. This usually only changes the chip if a top-up happened in the meantime, but it is computed and returned unconditionally so the terminal logic does not have to branch on whether a credit occurred.

4.4 Chip data layout and HMAC

The chips used for the bracelets are NTAG class tags, which are cheap and widely available, but do not have the cryptographic features of DESFire. They cannot do mutual authentication on their own, so the security has to come from a signed record written into a single user-memory page. The on-chip layout is small and fixed.

```
balance              uint32 little-endian (cents)
debit_counter        uint32 little-endian
credit_counter_seen  uint32 little-endian
hmac                 bytes(16)  HMAC-SHA256 truncated to 128 bits
                                over the 12 bytes above
```

The first twelve bytes are the state. The last sixteen bytes are an HMAC computed over those twelve bytes using a per-bracelet symmetric key. Anyone reading the chip can read the state, but only someone who knows the per-bracelet key can produce a record that the terminal will accept.

The per-bracelet key is derived rather than stored. A single master key lives behind the backend, and every bracelet's working key is computed as

```
key = HKDF(masterKey, salt = wristbandUid, info = "bracelet-v1")
```

The wristband's UID is read from the chip's read-only manufacturer area, so it cannot be spoofed by simply rewriting the user memory. The `info` string `"bracelet-v1"` fences the derivation against reuse if a future version of the layout ever needs to coexist with this one. In production the master key never leaves the festival HSM. For the demo it is provided to the backend through an environment variable, `BRACELET_SIGNING_KE` which keeps the development setup simple while leaving room for the HSM-backed deployment to drop in without touching application code.

The threat model around plain NFC, including replay of an old signed record after a top-up and physical cloning of the chip, is discussed in Chapter 5. The implementation choices here (the per-bracelet derived key, the inclusion of both counters inside the signed record, and the server-owned `credit_counter`) are all in service of the mitigations described there.

4.5 Online debit path

When a vendor terminal has connectivity, the simplest and safest thing to do is to push the debit to the server first and let the chip catch up afterwards. The terminal posts the debit to a charge endpoint that takes the bracelet, the amount, the vendor identity, and an idempotency key generated on the device. The server opens a serializable transaction on the bracelet row, checks that the balance covers the amount, decrements the balance, allocates the next `debit_counter` value (one above whatever was last applied), inserts a `transactions` row with that counter, and returns the new `balance` and `debit_counter` to the terminal.

The terminal then writes the new state back to the chip in a single NFC session, recomputing the HMAC over the new balance, debit counter, and the unchanged `credit_counter_seen`. If that write succeeds, the transaction is closed. If it fails because the customer pulled their wrist away too early, the server-side debit has already happened and the chip is now stale relative to the ledger. The next time the chip is read by any terminal, the server's view will be lower than the chip's view, and on the next sync the terminal can write the corrected state down. This is the same self-healing path the offline flow relies on, just hit from a different angle.

The decision to do server-first online and chip-first offline is intentional. When the network is up, the server is the cheapest place to enforce invariants and the chip is the more failure-prone half. When the network is down, the chip is the only place where the invariant can be enforced at all. The model swaps which side leads based on what is actually available.

4.6 Offline debit path and the queue

The offline path starts on the terminal. When connectivity is missing, or when the request to the charge endpoint times out, the terminal falls back to local authorisation. It reads the chip, verifies the HMAC against the per-bracelet key (which the terminal can derive itself, because it knows the master key for the duration of the event), and checks that the balance covers the amount.

If the check passes, the terminal performs the debit on the chip atomically inside a single NFC session. The new balance is computed (`balance - amount`), the debit counter is incremented by one, the credit counter seen is left untouched, the HMAC is recomputed over the three new fields, and all twenty-eight bytes are written back to the chip in one shot. The single-session write is important: if the customer pulls away halfway through, the chip is either fully updated or fully unchanged, never half written, because NTAG page writes are atomic at the page level and the layout was sized to fit cleanly into that constraint.

Only once the chip write is acknowledged does the terminal enqueue the pending debit in AsyncStorage. The queue entry carries the same fields the sync request will eventually need: a generated `idempotencyKey`, the `amount`, the `vendorId`, the `counterValue` that was just written to the chip, the `deviceId`, and the local timestamp. Storing the entry only after the chip write means the queue can never contain a debit that did not happen; the worst case is a chip update that the terminal forgot, which the next reconciliation will detect through the counter gap.

When connectivity comes back, the terminal posts the queue to the sync endpoint described in Section 4.3. The server replies with which entries it accepted, which it rejected, and what the chip should hold. The terminal removes the accepted entries from AsyncStorage. Rejected entries are kept and surfaced to the operator with the reason; `duplicate` is silently dropped, `insufficient_funds` and `counter_gap` are escalated to a manual review screen because they imply something happened that the automatic logic should not paper over.

4.7 Topup path

Top-ups are the credit side of the system, and they always go through the server because they involve real money moving through Stripe. The terminal (or the attendee's own phone, depending on the flow) creates a Stripe Payment Intent against the backend, which returns the client secret. The user confirms the payment in the Stripe SDK. On success, Stripe sends a `payment_intent.succeeded` webhook to the backend.

The webhook handler verifies the Stripe signature, looks up the corresponding `payment_intents` row by the intent ID, and runs the credit inside a serializable transaction on the target bracelet. The handler increments `bracelet.balance` by the topped-up amount, increments `bracelet.credit_counter` by one, and inserts a `transactions` row with that new `credit_counter` value and a reference to the payment intent. The serializable isolation level matters here for the same reason it does in the debit path: a webhook retry from Stripe and a manual reconciliation could otherwise both try to apply the same credit. Idempotency is enforced both by the unique constraint on the payment intent ID in `transactions` and by the counter check on the chip when the next sync happens.

After the credit is committed, the next time the bracelet meets a terminal, the sync response carries a `chipShouldWrite` that reflects the new balance and the new `credit_counter`. The terminal writes this back to the chip, and the chip's `credit_counter_seen` catches up to the server's `credit_counter`. From that point on, the chip and the server agree, and the bracelet is ready to be debited again, online or offline.

Chapter 5

Security Considerations

5.1 Authentication and Authorization

5.2 Wallet Integrity and Double-Spend Prevention

5.2.1 Online Mode

5.2.2 Offline Mode

5.3 Offline Trust Model

5.4 Stripe and Payment Security

5.5 Data Protection

Chapter 6

Discussion

This chapter looks back at the system we built and tries to answer the questions that were left open in the objectives. We talk about the trade-offs we made, how the system compares to what already exists on the market, what the current MVP does not cover, and what would need to happen for a real production deployment.

6.1 Design Trade-offs

6.1.1 Online by default, offline as a fallback

The first and probably most important decision was to treat offline as a bounded fallback and not as the foundation of the architecture. A lot of the academic work on local-first software [KWvHM19] pushes the idea that the local device should be the primary copy and that the cloud is just a sync target. For a payment system at a festival we think this is the wrong default. Money is a conserved quantity, not a piece of text that two people can edit and merge later. If two terminals approve a payment from the same wristband at the same time, you cannot just merge the two states and call it a day, you need to make sure the customer cannot spend the same balance twice.

The other reason is operational. Building the whole system around the assumption that the network will fail forces every component to deal with eventual consistency, queues, conflicts, and retries. That is a lot of code to write, test, and operate, for a problem that only happens during outages. We preferred to invest in keeping the network up, with Starlink as the primary uplink and a redundant topology, and to keep the offline path as a small, well-defined fallback that only does debits.

The trade-off is clear: when the network is up we have a simple architecture where the server is the source of truth, and when the network is down we accept a reduced feature set (no top-ups, no real-time analytics, no refunds) in exchange for keeping payments flowing. We think this is the right trade for a festival, where

outages are short and infrequent if the network is built properly, but the cost of a payment refusal during a peak hour is very high.

6.1.2 Server balance versus chip balance

The hybrid model splits the balance across two writers: the server owns credits, the chip owns debits. The natural alternative would be to keep a single authoritative balance on the server and let the chip carry only a cached copy that is overwritten on every tap. We considered this and rejected it, because as soon as the network is gone the cached copy is the only thing the terminal can read, and without a debit counter on the chip there is no way to detect double-spend across two terminals serving the same wristband during an outage.

The opposite extreme would be to keep the balance only on the chip, like a transit card, and let the server be a passive observer that learns about transactions through the sync stream. This is essentially what Suica and Octopus do (Section 2.1.1). It works for transit because credits also happen at a physical machine that writes to the card, but it does not work for our case because top-ups happen through Stripe in the app, with no chip nearby. The credit needs to land on the server first and then propagate to the chip on the next tap.

The two-counter model is a compromise between the two extremes. It gives us the simplicity of a server-owned balance for credits, the offline robustness of a chip-owned balance for debits, and a clean way to merge the two streams without conflict because each counter has exactly one writer.

6.1.3 Consistency versus availability during outages

The CAP theorem says that during a network partition you have to pick between consistency and availability, and we picked availability. Refusing a payment because the network is bad is not acceptable at a festival, so the chip approves the debit locally and we accept that the server will learn about it later. The risk is that two terminals could in theory approve more than the chip actually has, but because the chip is a single physical object that the customer carries, only one terminal can talk to it at a time. The atomic decrement on the chip itself is what saves us from double-spend, not any kind of distributed consensus.

The price we pay is that the server's view of the balance can lag during an outage, and that there is a small window where a terminal that dies before draining its queue can lose transactions. We discuss this in the next subsection.

6.1.4 Lost transactions and operator risk

The reconciliation model in Section 3.2 is correct as long as terminals eventually sync. If a terminal dies before it can push its queue, the chip subtracted the amount but the server has no record. The gap is detectable, because the chip's `debit_counter` is bigger than the server's `debit_counter_seen` plus the count of pending queued transactions, but the actual transaction details are gone with the device.

We treat this as an operational risk, not a correctness bug. The exposure is bounded by how often terminals sync (we sync every few minutes when online) and by how many transactions a single terminal can buffer before its battery dies. The festival absorbs the loss for those transactions, and vendors with local receipts can claim manually. In practice this is the same way cash works at a festival: if a vendor loses their cash box, the festival eats the loss. The difference is that we get a numerical bound on the maximum loss per terminal, which we did not have with cash.

6.1.5 Starlink and a redundant topology versus other connectivity options

We chose Starlink as the primary uplink because it gives us a fat pipe that does not depend on the local cellular network being able to handle a sudden density of devices. The Shafiq et al. [SJL+13] study is a good reminder of how badly cellular networks fall over when too many people gather in the same place. Starlink does not have this problem because the bandwidth is shared across a satellite footprint, not a single tower in a field.

The alternatives we considered were a private LTE deployment, an on-site mesh network between vendor terminals, or just relying on the venue's existing Wi-Fi. Private LTE is expensive and requires spectrum that is not always easy to get on short notice. A mesh network between phones is interesting in theory but the moment one device has the only path to the gateway, you get hot spots and weird routing problems. Venue Wi-Fi is rarely good enough for thousands of concurrent connections. Starlink with a redundant topology is not free either, but it is the most predictable option and it does not require the venue to have any infrastructure.

6.2 Comparison with Existing Solutions

6.2.1 Commercial festival platforms

Tappit, Intellitix, Weezevent, and Glownet have been deploying cashless systems for years. The architecture is mostly the same across them: RFID wristband, on-site

server, vendor terminals on a private network, and a back office that handles top-ups and reporting. They do this at a much bigger scale than what we built, with reliability that has been proven across hundreds of festivals.

Where our approach is different:

- We do not require an on-site server or an IT team at the venue. The backend is a single multi-tenant cloud service. Conversations about the Untold deployment confirmed that even with on-site infrastructure, top-up sync delays of two to four hours were common during the event.
- We treat the wristband as a cache, not a passive token. The balance and the debit counter live on the chip, so the terminal does not need to talk to a local server to authorize a payment.
- The architecture is documented and the code is open. The commercial systems are essentially black boxes in the academic literature, and the failures we know about (Download 2015, the Danish Nets outage in 2024) are reconstructed from press coverage and tweets.

Where the commercial systems are clearly ahead:

- Hardware. They use proper RFID wristbands with a secure element (DESFire EV2 or EV3), authenticated reads and writes, and tamper detection. Our MVP uses a plain NTAG-class chip without a secure element, which is fine for a thesis demo but not for production.
- Operations. They have on-site teams that can swap a broken reader in minutes, handle vendor onboarding at the gate, and deal with the long tail of weird customer issues. We do not have any of this.
- Scale. They have processed tens of millions of transactions across hundreds of events. We have a working prototype that has been tested in a development environment.

The honest summary is that we built a different architectural shape, not a better product. The commercial systems are the result of years of iteration on real festivals, and any open alternative would need a similar amount of time in the field to catch up on the operational side.

6.2.2 Transit cards

Transit cards (Suica, Octopus, Oyster) are the closest existing system to what we built, in the sense that they also store a balance on a chip and let a terminal make

a local decision. The big difference is the credit flow: at a transit station, the credit also happens at a physical machine that writes to the card, so the card is the source of truth for both directions. In a festival the credit happens through Stripe in the app, with no chip nearby, so the server has to be the source of truth for credits.

The other difference is the threat model. Transit fraud at the per-transaction level is small (a few euros), and the operators have built their cryptographic protocols around this. Festival transactions can be larger (a meal, a round of drinks for a group), so the financial incentive to tamper with a chip is bigger, and a production deployment would need a chip with a secure element. The lesson we take from transit is that the on-chip balance pattern works at scale, but the security properties have to be sized for the value that can be moved.

6.2.3 Open versus proprietary

The main contribution of this thesis is an open, documented architecture for a closed-loop festival payment system. The commercial systems work, but they do not help anyone who wants to understand how to build one, or who wants to deploy a similar system without paying license fees and signing a multi-year contract. Smaller festivals in particular cannot afford the commercial platforms, and they end up either staying on cash or going with a half-broken homegrown solution.

We hope that having an architecture written down, with the trade-offs discussed and the failure modes acknowledged, makes it easier for the next person to build on top of it without repeating the same mistakes.

6.3 Limitations

6.3.1 Hardware: a plain NFC chip is not enough for production

The biggest limitation of the current MVP is that the wristband is a plain NTAG-class NFC chip without a secure element. The hybrid model itself does not depend on chip-side cryptography, but the security properties of the system do. A motivated attacker with a cheap NFC writer can read the balance, write a higher balance, and walk back to the vendor. The server can detect this on the next sync (counters do not line up), but by then the customer has already left with the food. Section 5 in the security chapter lays out the upgrade path to a DESFire EV2 or EV3 chip with AES-128 mutual authentication, which is what a production deployment should use.

6.3.2 Offline duration is assumed to be short

The reconciliation model assumes that outages are short, on the order of minutes, not hours. The maximum amount that can be lost from a dead terminal scales with how long the terminal has been offline, and during a multi-hour outage a single terminal could buffer hundreds or thousands of transactions before its battery dies. We did not implement any limit on the queue size or any escalation path for prolonged outages. A production system would need to set a hard cap on offline operation and probably refuse new transactions once the queue gets above some threshold.

6.3.3 Single-event scope in testing

The architecture is multi-tenant by design (events are scoped through the `event_id` chain), but we only ran end-to-end tests against a single event at a time. We did not test what happens when two events are running simultaneously and share the same backend. The schema should handle it, but the operational side (load isolation, audit log scaling, admin dashboard performance) was not exercised.

6.3.4 No peer-to-peer transfers between attendees

The system does not support transfers between attendees. If two friends want to split a meal, one of them has to pay and the other has to send money back through some other channel. We could add it as an online-only operation through the app, but it adds a fraud surface (free money transfer through fake accounts) that we did not want to deal with for the MVP.

6.3.5 Refund flow is partial

Refunds for unspent balance at the end of an event are something every commercial system handles, and we only have a basic version of it. The server can credit the balance back to the original Stripe payment method through the standard Stripe refund API, but the policy questions (refund window, partial refunds, fees, refund of cash top-ups) are not implemented and are largely operational decisions that a real festival would have to make.

6.3.6 No real load testing

We argued in Section 3.7 that PostgreSQL on Neon should handle a festival comfortably, citing the OpenAI [Ope26] numbers. We did not actually run a load test that simulates a realistic festival traffic pattern (sharp peaks at meals and between

concerts, thousands of concurrent terminals). The numbers we have are good in theory, but a production deployment should run a load test against a realistic synthetic workload before the first event.

6.3.7 Vendor onboarding is a manual review

The current onboarding flow is that a vendor submits an application from the app, the admin reviews it manually, and approves or rejects it. This works for a festival with a couple of dozen vendors, but it does not scale to a venue with hundreds. A production system would need an automated vetting flow with KYC, document verification, and probably a tiered approval process.

6.4 Further Improvements

6.4.1 Move to authenticated NFC chips

The first thing a production deployment should do is move from plain NTAG chips to DESFire EV2 or EV3 with AES-128 mutual authentication. This closes the obvious tampering attack and brings the security level up to what transit systems and commercial festival platforms already have. The hybrid reconciliation model does not change at all, only the read and write paths on the terminal side need to do the cryptographic handshake.

6.4.2 Fraud detection on the server

Even with authenticated chips, server-side anomaly detection would catch things that the chip cannot see on its own. Examples: a wristband that suddenly debits at two terminals on opposite sides of the venue within a minute (cloning), a vendor whose ratio of offline to online debits is much higher than the average (possible terminal manipulation), or a sudden spike in average transaction amount on a specific terminal. None of this exists in the MVP and all of it would be valuable in production.

6.4.3 Better offline strategies

If the festival operator wanted an extra layer of resilience, the vendor terminals could form a mesh network between themselves so that one terminal with connectivity could relay sync traffic for a small group of nearby terminals during an outage.

We deliberately did not build this for the MVP because the chip-as-cache model already covers the common case, but for very large venues with patchy coverage it could reduce the worst-case offline duration per terminal.

6.4.4 Real-time analytics for the operator

The admin dashboard currently shows transaction volume and basic stats, but it does not have the kind of real-time analytics that festival operators want during the event (top-selling vendors, queue lengths approximated by debit rate, top-up rate per gate, anomaly alerts). A production deployment would need a streaming analytics pipeline (probably something like ClickHouse or Materialize on top of the transaction log) to give operators the visibility they need to make decisions during the event.

6.4.5 Multi-event support and tenant isolation

The schema is multi-tenant but we did not test what happens when two big events run on the same deployment at the same time. Going to production would mean exercising this path: separate audit log streams per event, per-event rate limiting, per-event database quotas, and probably a way to spin up dedicated read replicas for very large events without changing application code.

6.4.6 Vendor settlement and payout automation

At the end of an event, vendors expect to get paid based on their sales minus the festival commission. The current system records all transactions with the vendor and commission attached, but the actual payout (computing the totals, deducting fees, sending money to the vendor's bank account through Stripe Connect) is a manual operation. Automating this with Stripe Connect and a settlement run that the admin can review and confirm would close one of the biggest operational gaps.

6.4.7 Integration with other festival systems

The wristband is already a piece of hardware that the attendee carries, and it could be used for more than payments: access control at gates, age verification at bars, VIP area access, queue-skip features. The data model already has the wristband as a first-class entity, and adding a permissions layer per wristband would let the same hardware drive several festival systems from a single source of truth. This is what some commercial platforms already do, and it is a natural extension of the current architecture.

Chapter 7

Conclusions

Bibliography

[CP03] Patrick Y. K. Chau and Simpson Poon. Octopus: An E-cash payment system success story. *Communications of the ACM*, 46(9):129–133, 2003.

[DHJ+07] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP 2007)*, pages 205–220, 2007.

[eFe16] eFestivals. Download festival scraps on site cashless only payment system. `https://www.efestivals.co.uk/news/16/160301h.shtml`, 2016. Online; accessed 2026.

[Eur09] European Parliament and Council of the European Union. Directive 2009/110/EC on the taking up, pursuit and prudential supervision of the business of electronic money institutions. `https://eur-lex.europa.eu/eli/dir/2009/110/oj/eng`, 2009. Online; accessed 2026.

[Eur15] European Parliament and Council of the European Union. Directive (eu) 2015/2366 on payment services in the internal market (PSD2). `https://eur-lex.europa.eu/eli/dir/2015/2366/oj/eng`, 2015. Online; accessed 2026.

[Fow05] Martin Fowler. Event sourcing. `https://martinfowler.com/eaaDev/EventSourcing.html`, 2005. Online; accessed 2026.

[GdKGM+08] Flavio D. Garcia, Gerhard de Koning Gans, Ruben Muijrers, Peter van Rossum, Roel Verdult, Ronny Wichers Schreur, and Bart Jacobs. Dismantling MIFARE classic. In *Proceedings of the 13th European Symposium on Research in Computer Security (ESORICS 2008)*, volume 5283 of *LNCS*, pages 97–114. Springer, 2008.

[GL02] Seth Gilbert and Nancy Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.

[KB17] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. Technical Report 10, 2017.

[KWvHM19] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: you own your data, in spite of the cloud. In *Proceedings of ACM Onward! 2019*, pages 154–178, 2019.

[MDAB10] Steven J. Murdoch, Saar Drimer, Ross Anderson, and Mike Bond. Chip and PIN is broken. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, pages 433–446, 2010.

[Ope26] OpenAI. Scaling PostgreSQL to power 800 million ChatGPT users. `https://openai.com/index/scaling-postgresql/`, 2026. Online; accessed April 2026.

[Rie11] Eric Ries. *The Lean Startup*. Crown Business, New York, 2011.

[SBH24] Lachlan Schomburgk, Alessandro Belli, and Arvid O. I. Hoffmann. Less cash, more splash? A meta-analysis on the cashless effect. *Journal of Retailing*, 100(3):382–403, 2024.

[SCZ+16] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.

[SDK24] SDK.finance. Closed-loop payment systems: Definition, benefits, and business cases. `https://sdk.finance/blog/closed-loop-payment-systems-definition-benefits-and-business` 2024. Online; accessed 2026.

[Shi02] Yasutomo Shirakawa. JR East contactless IC card automatic fare collection system "Suica". In *Proceedings of the 7th IEEE International Symposium on High Assurance Systems Engineering (HASE 2002)*, pages 3–10, 2002.

[SJL+13] Muhammad Zubair Shafiq, Lusheng Ji, Alex X. Liu, Jeffrey Pang, Shobha Venkataraman, and Jia Wang. A first look at cellular network performance during crowded events. In *Proceedings of the ACM SIGMETRICS 2013*, pages 17–28, 2013.

[Son24a] Sony Corporation. Case study: Hong Kong octopus card. https://www.sony.co.jp/en/Products/felica/casestudy/article/octopus.html, 2024. Online; accessed 2026.

[Son24b] Sony Corporation. FeliCa – contactless IC card technology. https://www.sony.net/Products/felica/about/, 2024. Online; accessed 2026.

[SPBZ11] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, pages 386–400, 2011.

[Tap24] Tappit. RFID cashless payments for events. https://tappit.com/pay/rfid, 2024. Online; accessed 2026.

[Vog09] Werner Vogels. Eventually consistent. *Communications of the ACM*, 52(1):40–44, 2009.

[You10] Greg Young. CQRS documents. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf, 2010. Online; accessed 2026.